



MODELACIÓN NUMÉRICA
CURSO TARELA

Trabajo Práctico de Máquina Artemis II



8 de junio de 2026

Tomás Conti
111760

Agustín Dubovitsky Otero
111954

Índice

1. Consignas	3
1.1. Órbita Lunar	3
1.2. Trayectoria de Orion	3
2. Introducción	4
3. Desarrollo	4
3.1. Órbita Lunar	4
3.1.1. Discretización de EDOs	4
3.1.2. Condiciones iniciales del PVI	4
3.1.3. Elección del paso temporal	5
3.2. Trayectoria de Orion	6
3.2.1. Discretización de EDOs	6
3.2.2. Condiciones Iniciales	6
3.2.3. Elección del Paso Temporal	6
4. Resultados y Comparación de Euler y RK2	8
4.0.1. Discretización de EDOs	8
4.1. Estudio de la órbita lunar: Método de Euler vs. Runge-Kutta de 2do Orden (RK2)	8
4.1.1. Caso 1: Paso de tiempo pequeño ($dt = 60$ s)	8
4.1.2. Caso 2: Paso de tiempo grande ($dt = 43200$ s / 12 horas)	9
4.2. Simulación de la Trayectoria de la Nave Orion	11
5. Conclusiones	11
6. Referencias	12
7. Códigos	12
7.1. Código Discretizaciones	12
7.2. Código trayectoria Lunar	12
7.3. Código trayectoria Orion	15

1. Consignas

1.1. Órbita Lunar

La trayectoria de la nave se ve fuertemente afectada por el campo gravitatorio de la Tierra y de la Luna. Para ello, se necesita saber la posición de la Luna en todo momento. Asumiendo posición inicial y velocidad, se puede calcular la trayectoria con la siguiente ecuación diferencial:

$$\frac{d^2\vec{x}}{dt^2} = \vec{a}$$

Donde $\vec{x}$ es la posición vectorial de la Luna y $\vec{a}$ la aceleración producida por el campo gravitatorio terrestre. Si escribimos la posición y la aceleración en dos dimensiones (x e y), y descomponemos la ecuación de segundo orden en dos ecuaciones de primer orden, nos queda:

$$\frac{dx}{dt} = v_x$$

$$\frac{dy}{dt} = v_y$$

$$\frac{dv_x}{dt} = G \frac{M_T}{d_T^2} \cos \alpha$$

$$\frac{dv_y}{dt} = G \frac{M_T}{d_T^2} \sin \alpha$$

donde G es la constante de gravitación universal, M_T la masa de la Tierra, $d_T = \sqrt{x^2 + y^2}$ la distancia entre la Luna y la Tierra, $\alpha = \text{atan2}(y, x)$ el ángulo que forma el vector posición con el eje x , y h el paso temporal.

1.2. Trayectoria de Orion

Con las mismas ecuaciones, y conociendo la posición de la Luna para todo tiempo t , calcular la posición de Orion desde la fase 10 de la figura 2 hasta la fase 13. Durante todo este tiempo, la nave está (casi) en una trayectoria libre, mayormente afectada por la gravedad de la tierra y la luna. Tomar una posición y velocidad inicial aproximada, sacándolo de los datos de la telemetría, entre las 4 y las 6 am (UTC) del 3 de abril.

$$\frac{dx}{dt} = v_x$$

$$\frac{dy}{dt} = v_y$$

$$\frac{dv_x}{dt} = G \frac{M_T}{d_T^2} \cos \alpha + G \frac{M_L}{d_L^2} \cos \beta$$

$$\frac{dv_y}{dt} = G \frac{M_T}{d_T^2} \sin \alpha + G \frac{M_L}{d_L^2} \sin \beta$$

donde G es la constante de gravitación universal, M_T la masa de la Tierra, M_L la masa de la Luna, d_T la distancia entre la nave y la Tierra, d_L la distancia entre la nave y la Luna, α el ángulo que forma el vector posición con el eje x respecto a la Tierra, β el ángulo respecto a la Luna, y h el paso temporal.

2. Introducción

En este trabajo, se aborda la modelización de la trayectoria de la nave Orión en la misión Artemis II, mediante la aplicación de métodos numéricos. El objetivo principal es comparar el comportamiento observado del modelo teórico, con el de los experimentos.

La simulación se realizó con tres métodos de distinto orden, con fin de compararlos y observar el impacto que se genera en los resultados al realizar cambios en los valores iniciales.

El trabajo consta de dos partes. La primera, se centra en la obtención de la órbita de la luna sobre la tierra, y la segunda, en simular la trayectoria de orión en su recorrido entre la tierra y la luna.

3. Desarrollo

En esta sección se presentan los detalles del proceso de modelización de la trayectoria de la nave Orión, junto con los criterios adoptados en cada etapa.

3.1. Órbita Lunar

Como paso previo a la simulación de la trayectoria de Orión, se modeló la órbita de la Luna alrededor de la Tierra, para conocer la posición lunar en todo instante.

3.1.1. Discretización de EDOs

Para resolver las ecuaciones dadas por el método de **Runge-Kutta 4 (RK4)** mediante PVI se discretizaron las ecuaciones de la siguiente forma:

$$\vec{q}_1 = \vec{f}(\vec{u}_n)$$

$$\vec{q}_2 = \vec{f}\left(\vec{u}_n + \frac{h}{2} \vec{q}_1\right)$$

$$\vec{q}_3 = \vec{f}\left(\vec{u}_n + \frac{h}{2} \vec{q}_2\right)$$

$$\vec{q}_4 = \vec{f}(\vec{u}_n + h \vec{q}_3)$$

$$\vec{u}_{n+1} = \vec{u}_n + \frac{h}{6} (\vec{q}_1 + 2\vec{q}_2 + 2\vec{q}_3 + \vec{q}_4)$$

con $\vec{f}(\vec{u}) = \left(v_x, v_y, -G \frac{M_T}{d_T^2} \cos \alpha, -G \frac{M_T}{d_T^2} \sin \alpha\right)$ el campo vectorial del sistema.

3.1.2. Condiciones iniciales del PVI

Se coloca a la Luna en el **-perigeo** como condición inicial. En ese instante el vector posición es puramente horizontal y la velocidad es perpendicular al radio:

$$x_0 = R_{\text{perigeo}}, \quad y_0 = 0, \quad v_{x,0} = 0, \quad v_{y,0} = v_{\text{perigeo}}$$

Los valores de perigeo y apogeo se obtienen de **NASA horizon system**:

$$R_{\text{perigeo}} = 356\,500 \text{ km} = 3,565 \times 10^8 \text{ m}, \quad R_{\text{apogeo}} = 406\,700 \text{ km} = 4,067 \times 10^8 \text{ m}$$

La velocidad en el perigeo se obtiene de la **ecuación de la vis-viva**:

$$v = \sqrt{GM_T \left(\frac{2}{r} - \frac{1}{a} \right)}$$

donde a es el semieje mayor de la órbita elíptica:

$$a = \frac{R_{\text{perigeo}} + R_{\text{apogeo}}}{2} = 3,816 \times 10^8 \text{ m}$$

Evaluyendo en $r = R_{\text{perigeo}}$ con las constantes $G = 6,674 \times 10^{-11} \text{ m}^3 \text{ kg}^{-1} \text{ s}^{-2}$ y $M_T = 5,972 \times 10^{24} \text{ kg}$:

$$v_{\text{perigeo}} = \sqrt{GM_T \left(\frac{2}{R_{\text{perigeo}}} - \frac{1}{a} \right)} \approx 1,076 \times 10^3 \text{ m/s}$$

Resumiendo, el vector de estado inicial es:

$$\mathbf{q}(0) = \begin{pmatrix} -3,565 \times 10^8 \text{ m} \\ 0 \\ 0 \\ -1,076 \times 10^3 \text{ m/s} \end{pmatrix}$$

Se utilizaron los valores en negativo para que orion encuentre a la luna, dado que si la luna empieza del otro extremo de la órbita, no va a llegar a tiempo para cuando orion pase.

3.1.3. Elección del paso temporal

Para obtener la trayectoria de una vuelta completa de la Luna en torno a la Tierra, se utilizó una duración de $T = 27,32$ días, lo cual es el tiempo oficial. El paso de tiempo h se eligió de forma que el número de pasos $N = T/h$ sea suficientemente grande para que el error de truncamiento local no acumule un error global apreciable. Se adoptó $h = 60\text{s}$, que garantiza al menos $N \approx 39,000$ puntos por órbita. Este nivel de discretización es suficiente para resolver con precisión las variaciones de posición y velocidad a lo largo de la trayectoria.

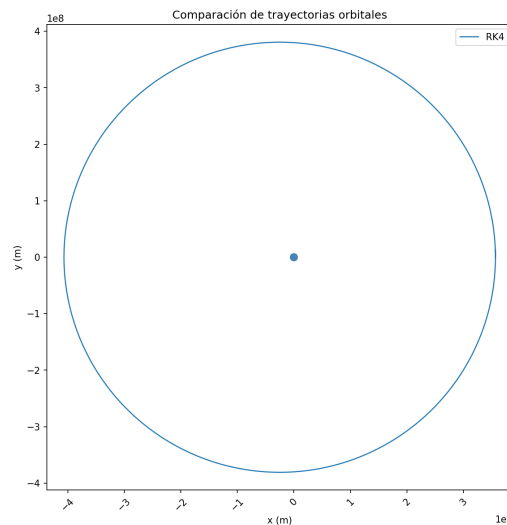


Figura 1: una vuelta exacta de la luna con el método RK4

3.2. Trayectoria de Orion

Habiendo calculado la órbita de la luna alrededor de la tierra, podemos calcular la trayectoria de orión para que de una vuelta por la luna y vuelva, manipulando el ángulo en el que empieza orion con fin de que se encuentre con la luna en algún momento de su trayectoria anteriormente calculada.

3.2.1. Discretización de EDOs

Para resolver las ecuaciones mediante **Runge-Kutta 4 (RK4)** se define el vector de estado $\vec{u}_n = (x_n, y_n, v_{x_n}, v_{y_n})$ y se discretiza como:

$$\vec{q}_1 = \vec{f}(\vec{u}_n)$$

$$\vec{q}_2 = \vec{f}\left(\vec{u}_n + \frac{h}{2} \vec{q}_1\right)$$

$$\vec{q}_3 = \vec{f}\left(\vec{u}_n + \frac{h}{2} \vec{q}_2\right)$$

$$\vec{q}_4 = \vec{f}(\vec{u}_n + h \vec{q}_3)$$

$$\vec{u}_{n+1} = \vec{u}_n + \frac{h}{6} (\vec{q}_1 + 2\vec{q}_2 + 2\vec{q}_3 + \vec{q}_4)$$

con $\vec{f} = \left(v_x, v_y, -G \frac{M_T}{d_T^2} \cos \alpha - G \frac{M_L}{d_L^2} \cos \beta, -G \frac{M_T}{d_T^2} \sin \alpha - G \frac{M_L}{d_L^2} \sin \beta\right)$, campo vectorial que representa el sistema de EDOs, es decir, las derivadas del vector de estado respecto al tiempo.

3.2.2. Condiciones Iniciales

Las condiciones iniciales de posición y velocidad de la nave Orion se obtuvieron de los datos de telemetría a la hora 05:03:39 del 3 de abril. Los valores utilizados son:

$$x_0 = -52409,924647197156 * 1000 \text{ m}$$

$$y_0 = -48671,635720228245 * 1000 \text{ m}$$

$$v_{x_0} = -1,22735334971968 * 1000 + extra_v[0] \text{ m/s}$$

$$v_{y_0} = -2,34643943970753 * 1000 + extra_v[1] \text{ m/s}$$

$extra_v[0]$ y $extra_v[1]$ son las desviaciones realizadas sobre el ángulo de dirección de orión, para lograr que se encuentre con la luna, de la vuelta, y vuelva a la tierra.

Las condiciones iniciales del vector de velocidad resultaron de una magnitud de 3022,74 m/s y una orientación de $-127,98^\circ$.

$$\vec{v}_0 = (3022,74 \cdot \cos(-127,98^\circ), 3022,74 \cdot \sin(-127,98^\circ)) \text{ m/s}$$

3.2.3. Elección del Paso Temporal

Para la trayectoria de orion se y la órbita lunar a la hora de resolver esta trayectoria, se volvió a utilizar como paso temporal $h = 60\text{s}$, para que tengan coherencia y buena precisión.

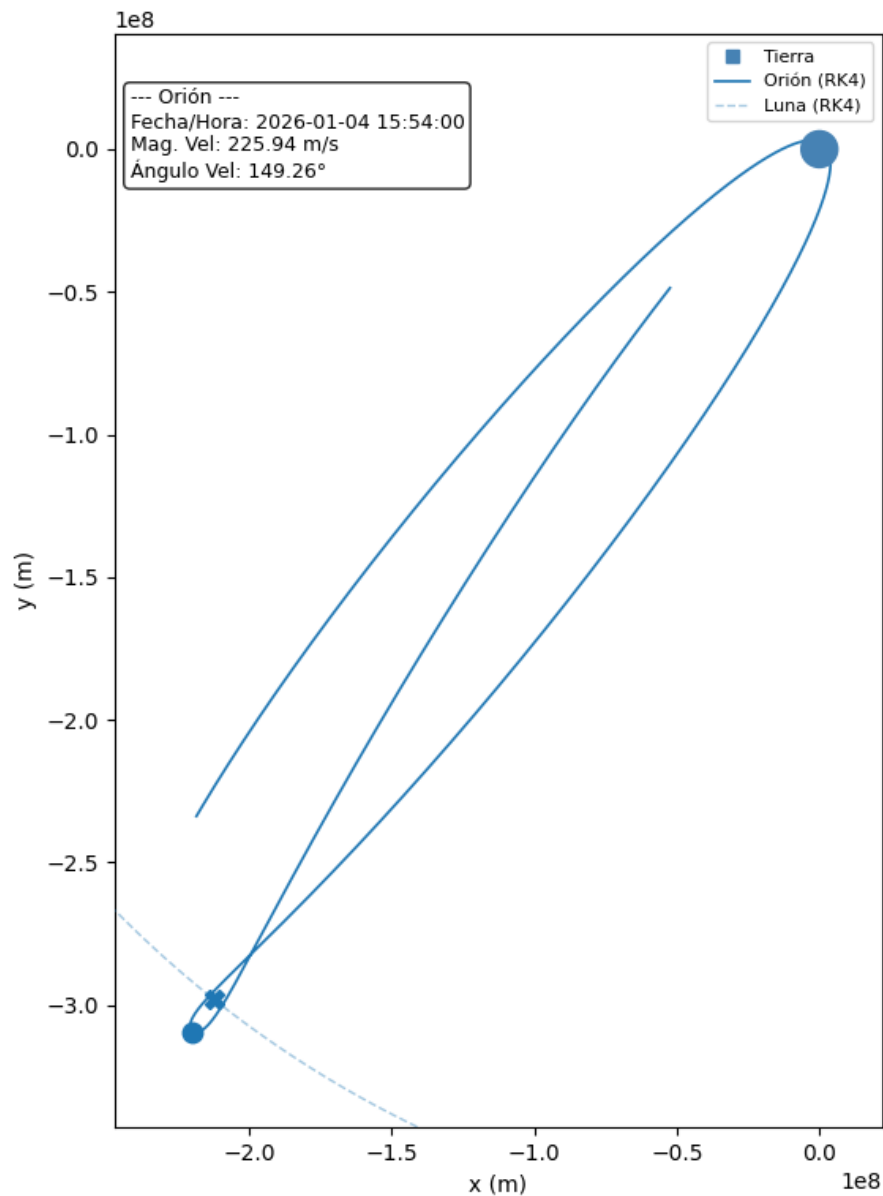


Figura 2: recorrido de ori3n con RK4

Se decidi3 utilizar un lapso de tiempo de 10 d3as, para asegurar que se complete el recorrido, y como podr3 ver en la im3gen, ori3n luego de dar la vuelta a la luna, volvi3 a la tierra y sigui3, lo cual es interesante pero para esta soluci3n no tiene relevancia. En las pr3ximas secciones, esto se utilizar3 para poder encontrar algunas diferencias entre los distintos m3todos utilizados.

4. Resultados y Comparación de Euler y RK2

En esta sección se presentan y analizan los resultados de las simulaciones numéricas para la órbita de la Luna y la trayectoria de la nave Orion, calculados con Euler y RK2. Para todas las simulaciones de la Luna se utilizaron como condiciones iniciales los valores correspondientes a su perigeo.

4.0.1. Discretización de EDOs

Euler Explícito Dado el sistema de ecuaciones de primer orden, la discretización por el método de Euler explícito resulta:

$$\begin{aligned}x_{n+1} &= x_n + h v_{x,n} \\y_{n+1} &= y_n + h v_{y,n} \\v_{x,n+1} &= v_{x,n} + h G \frac{M_T}{d_{T,n}^2} \cos \alpha_n \\v_{y,n+1} &= v_{y,n} + h G \frac{M_T}{d_{T,n}^2} \sen \alpha_n\end{aligned}$$

donde h es el paso de integración y n el índice del paso temporal.

Runge-Kutta de orden 2 (RK2) Definiendo $\vec{u} = (x, y, v_x, v_y)^T$ y $\vec{f}(\vec{u})$ como el lado derecho del sistema, las etapas del método son:

$$\begin{aligned}\vec{q}_1 &= \vec{f}(\vec{u}_n) \\ \vec{q}_2 &= \vec{f}(\vec{u}_n + h \vec{q}_1) \\ \vec{u}_{n+1} &= \vec{u}_n + \frac{h}{2} (\vec{q}_1 + \vec{q}_2)\end{aligned}$$

4.1. Estudio de la órbita lunar: Método de Euler vs. Runge-Kutta de 2do Orden (RK2)

Con el objetivo de evaluar la estabilidad y precisión de los integradores numéricos frente al paso de tiempo (dt), se realizaron simulaciones bajo dos escenarios distintos para los métodos de Euler y RK. Para todos los casos se proyectó la órbita durante un tiempo total de 100 órbitas.

4.1.1. Caso 1: Paso de tiempo pequeño ($dt = 60$ s)

En este primer escenario se utilizó un diferencial de tiempo de 60 segundos. Los resultados de las trayectorias orbitales obtenidas se muestran en la Figura 3.

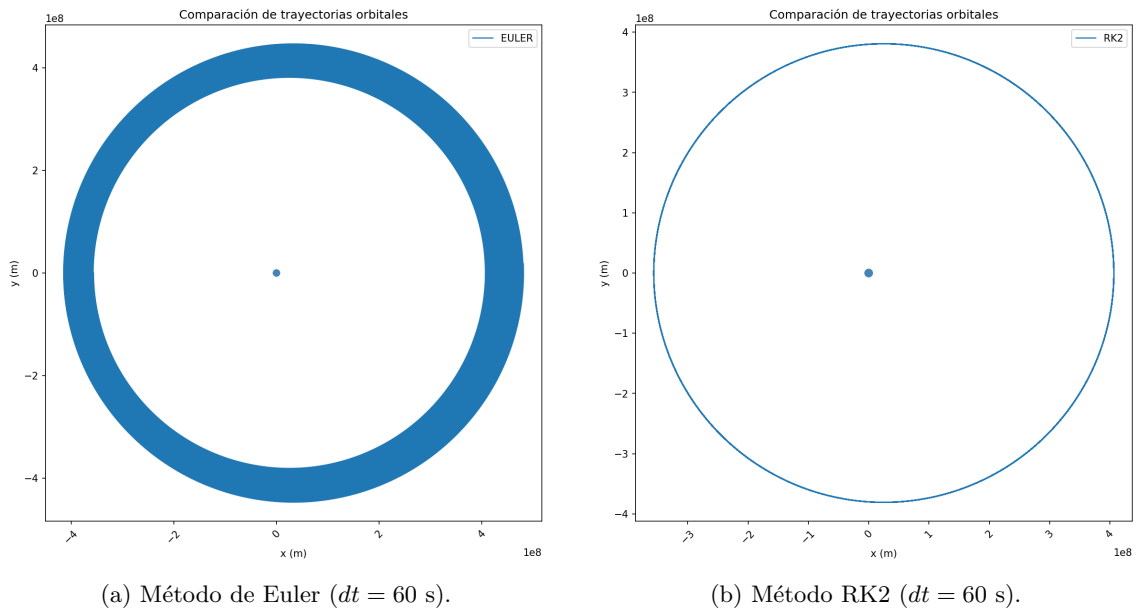


Figura 3: Comparación de trayectorias orbitales de la Luna tras 100 vueltas con un paso de tiempo corto.

Al observar la Figura 3a, se evidencia que el método de Euler presenta una acumulación de error numérica visible, manifestada como un ensanchamiento artificial de la órbita (pérdida de conservación de energía). A pesar de utilizar un paso de tiempo pequeño, el método diverge gradualmente hacia el exterior. Por el contrario, el método RK2 (Figura 3b) muestra una estabilidad notable, manteniendo la trayectoria concéntrica y cerrada a lo largo de las 100 órbitas simuladas, demostrando una mayor precisión en el orden de convergencia.

4.1.2. Caso 2: Paso de tiempo grande ($dt = 43200$ s / 12 horas)

Para evaluar la robustez de los algoritmos ante pasos de tiempo macroscópicos, se incrementó el valor de dt a 12 horas. Los resultados se exponen en la Figura 4.

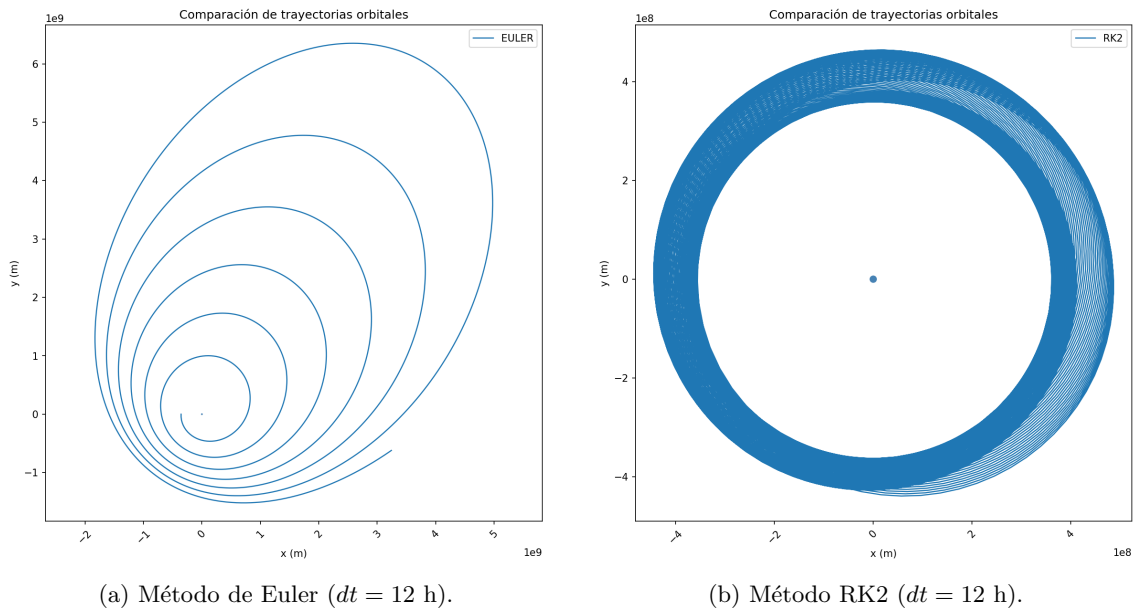


Figura 4: Comparación de trayectorias orbitales de la Luna tras 100 vueltas con un paso de tiempo de 12 horas.

Bajo estas condiciones extremas, el método de Euler (Figura 4a) colapsa por completo, describiendo una espiral divergente que aleja la Luna de su centro de gravedad de manera irreal.

Por otra parte, el método RK2 (Figura 4b) logra sostener la simulación sin divergir al infinito. Sin embargo, debido al excesivo tamaño de dt , se aprecia un fenómeno de precesión numérica y una oscilación en el radio orbital que ensancha la trayectoria. Esto demuestra que, si bien RK2 es drásticamente más estable que Euler, sigue estando sujeto a restricciones de discretización temporal para preservar la física real del sistema.

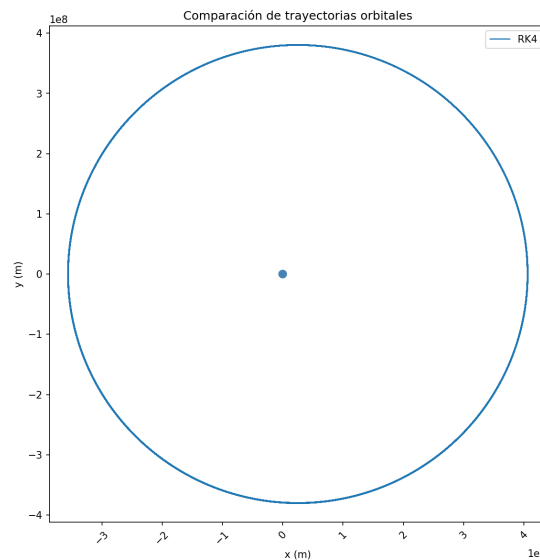


Figura 5: Método de RK4 ($dt = 12$ h).

Para el caso de la órbita lunar calculada con RK4, se puede ver que mantiene la estabilidad, incluso con un alto paso de tiempo, tras 100 vueltas, demostrando una mayor precisión.

4.2. Simulación de la Trayectoria de la Nave Orion

Habiendo determinado el comportamiento aislado del sistema Tierra-Luna, se procedió a simular la dinámica de la nave Orion interactuando bajo el campo gravitatorio de ambos cuerpos celestes. Para asegurar que la posición del entorno de referencia fuera lo más precisa posible, la órbita de la Luna se integró con el método RK2 utilizando un paso de tiempo óptimo de $dt = 60$ s durante un período de 1 vuelta completa.

La nave Orion se simuló durante un tiempo total de 10 días empleando el mismo diferencial de tiempo ($dt = 60$ s).

Con el fin de contrastar la respuesta de la trayectoria frente a los integradores numéricos analizados, se evaluó la misión comparando los esquemas de Euler y Runge-Kutta de 2do Orden (RK2). Los resultados se presentan en la Figura 6.

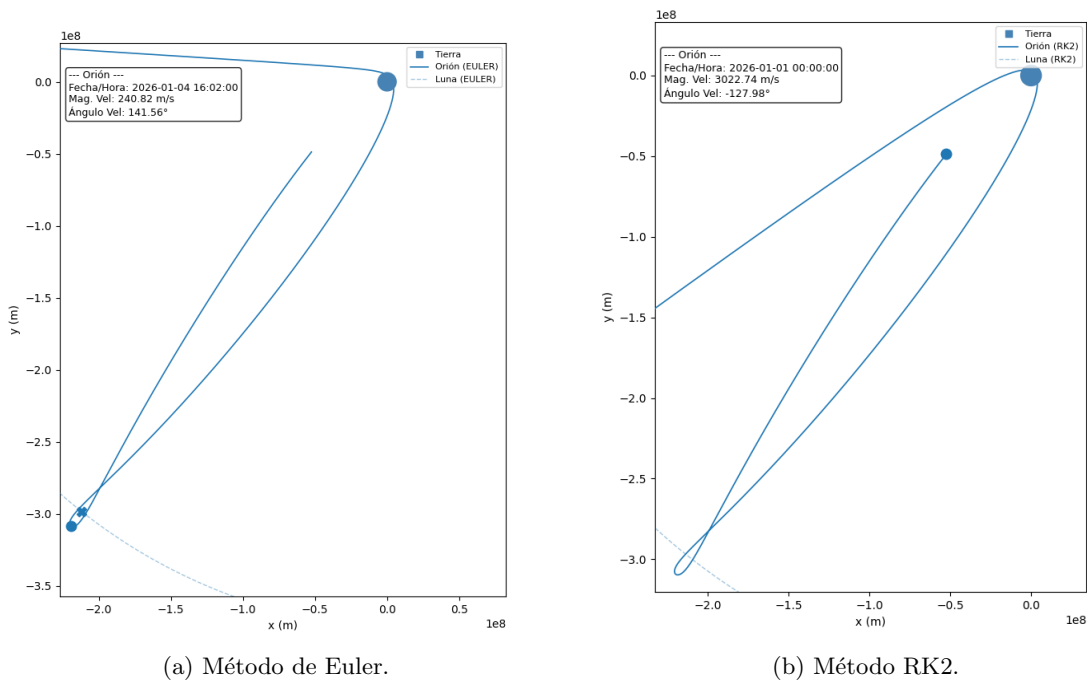


Figura 6: Trayectorias simuladas de la nave Orion en el sistema Tierra-Luna a lo largo de 10 días ($dt = 60$ s).

5. Conclusiones

Luego de haber realizado el experimento con distintos métodos, se logró entender a mayor profundidad el efecto del error de truncamiento en los métodos para PVI. Este error viene en conjunto con los valores iniciales utilizados, principalmente el valor del paso y el tiempo de duración, donde dependiendo del método, cuanto más se agrandaban los valores, más se alejaban de la solución real, tal y como se vio en clase.

Por otro lado, resultó muy interesante encontrar el ángulo de velocidad inicial de orión de forma tal que la nave se encuentre con la luna y realice el recorrido correcto, ya que se pudieron observar muchos factores que afectaban la trayectoria de la nave, ya sea la fuerza que ejerce la Luna y la Tierra, lo que sucede si Orion chocaba con alguno de estos cuerpos, o si simplemente Orion no se encontraba con la Luna o a la vuelta con la Tierra.

Un método alternativo para resolver este trabajo, el cuál mejore el comportamiento en relación a Runge-Kutta 2 y Euler Explícito podría ser el método conservativo de Nystrom de tres etapas, el cuál es un método de orden 4.

Trabajar sobre la misión Artemis II resultó en una buena experiencia debido a que fue un hecho reciente e interesante, del cuál resultó un aprendizaje más allá del análisis numérico.

6. Referencias

NASA horizon system: <https://ssd.jpl.nasa.gov/horizons/>

7. Códigos

7.1. Código Discretizaciones

```
1 def paso_euler(f, t, estado, h):
2     return estado + h * f(t, estado)
3
4
5 def paso_rk4(f, t, estado, h):
6     q1 = f(t, estado)
7     q2 = f(t + h/2, estado + h/2 * q1)
8     q3 = f(t + h/2, estado + h/2 * q2)
9     q4 = f(t + h, estado + h * q3)
10    return estado + (h / 6) * (q1 + 2*q2 + 2*q3 + q4)
11
12
13 def paso_rk2(f, t, estado, h):
14     q1 = f(t, estado)
15     q2 = f(t + h/2, estado + h/2 * q1)
16     return estado + h * q2
17
18
19 INTEGRADORES = {
20     'euler': paso_euler,
21     'rk2': paso_rk2,
22     'rk4': paso_rk4,
23 }
```

7.2. Código trayectoria Lunar

```
1 import argparse
2 import os
3 import sys
4
5 import numpy as np
6 import pandas as pd
7 import matplotlib
8 matplotlib.use('Agg')
9 import matplotlib.pyplot as plt
10 from matplotlib.patches import Circle
11
12 from discretizaciones import INTEGRADORES
13
14 G = 6.674e-11
15 M_T = 5.972e24
16
17 R_PERIGE0 = 3.565e8
18 R_APOGEO = 4.067e8
19 A_SEMIEJE = (R_PERIGE0 + R_APOGEO) / 2
20 V_PERIGE0 = np.sqrt(G * M_T * (2 / R_PERIGE0 - 1 / A_SEMIEJE))
21
22 EARTH_RADIUS_KM = 6371
23
24 PALETA = [
25     '#1f77b4',
```

```
26     '#d62728',
27     '#2ca02c',
28     '#9467bd',
29     '#8c564b',
30     '#e377c2',
31 ]
32
33 def derivadas(t, estado):
34     x, y, vx, vy = estado
35     d2 = x**2 + y**2
36     d = np.sqrt(d2)
37     ax = -G * M_T / d2 * (x / d)
38     ay = -G * M_T / d2 * (y / d)
39     return np.array([vx, vy, ax, ay])
40
41 def simular(metodo: str, dt: float, duracion_dias: float) -> pd.DataFrame:
42     integrador = INTEGRADORES[metodo]
43     t_total = duracion_dias * 86400.0
44     pasos = int(t_total / dt)
45
46     estado = np.array([-R_PERIGE0, 0.0, 0.0, -V_PERIGE0])
47     t = 0.0
48
49     registros = np.empty((pasos + 1, 5))
50     registros[0] = [t, estado[0], estado[1], estado[2], estado[3]]
51
52     print(f" [{metodo.upper()}] dt={dt}s | {duracion_dias} d as | {pasos:,} pasos ...", end=' ')
53
54     for i in range(pasos):
55         estado = integrador(derivadas, t, estado, dt)
56         t += dt
57         registros[i + 1] = [t, estado[0], estado[1], estado[2], estado[3]]
58
59     df = pd.DataFrame(registros, columns=['time_s', 'x', 'y', 'vx', 'vy'])
60     print(f"listo ({len(df):,} filas)")
61     return df
62
63 def cargar_csv(ruta: str) -> pd.DataFrame:
64     with open(ruta, 'r') as f:
65         lineas = f.readlines()
66
67     sep = ';' if any('; ' in l for l in lineas[:5]) else ','
68
69     if sep == ';':
70         df = pd.read_csv(ruta, sep=';', decimal=',', header=None,
71                          names=['time_s', 'x', 'y', 'z', 'vx', 'vy', 'vz'])
72     else:
73         header_idx = None
74         for i, linea in enumerate(lineas):
75             partes = [p.strip().upper() for p in linea.split(',')]
76             if 'X' in partes and 'Y' in partes:
77                 header_idx = i
78                 break
79
80         if header_idx is None:
81             print(f"Error: no se encontr encabezado con columnas X e Y en '{ruta}'")
82             file=sys.stderr)
83             sys.exit(1)
84
85         df = pd.read_csv(ruta, sep=',', header=header_idx, low_memory=False)
86
87         col_map = {}
88         for col in df.columns:
89             cu = col.strip().upper()
90             if cu in ('JDTDB', 'TIME_S', 'TIME'): col_map[col] = 'time_s'
91             elif cu == 'X': col_map[col] = 'x'
92             elif cu == 'Y': col_map[col] = 'y'
93             elif cu == 'VX': col_map[col] = 'vx'
```

```
94         elif cu == 'VY':                                col_map[col] = 'vy'
95         df = df.rename(columns=col_map)
96
97     for c in ['x', 'y']:
98         df[c] = pd.to_numeric(df[c], errors='coerce')
99
100    df = df.dropna(subset=['x', 'y'])
101
102    if df['x'].abs().max() < 1e7:
103        df['x'] *= 1e3
104        df['y'] *= 1e3
105
106    print(f" [CSV] {os.path.basename(ruta)} cargado ({len(df):,} filas)")
107    return df
108
109 def resolver_fuentes(metodos, dt, duracion, exportar_csv=True):
110     resultados = []
111     for m in metodos:
112         if m.startswith('csv:'):
113             ruta = m[4:]
114             if not os.path.exists(ruta):
115                 print(f"Error: no se encontr {ruta}", file=sys.stderr)
116                 sys.exit(1)
117             df = cargar_csv(ruta)
118             etq = os.path.splitext(os.path.basename(ruta))[0]
119         elif m in INTEGRADORES:
120             df = simular(m, dt, duracion)
121             etq = m.upper()
122         if exportar_csv:
123             os.makedirs("resultados", exist_ok=True)
124             nombre_csv = f"resultados/resultado_{etq.lower()}.csv"
125             df.to_csv(nombre_csv, index=False)
126             print(f" [{etq}] Resultados guardados en: {nombre_csv}")
127         else:
128             print(f"Error: m todo desconocido '{m}'. Opciones: rk4, euler, csv:<
129             ruta>",
130                   file=sys.stderr)
131             sys.exit(1)
132             resultados.append((etq, df))
133     return resultados
134
135 def graficar(fuentes, salida, dpi):
136     fig, ax = plt.subplots(figsize=(8, 8))
137
138     # Tierra
139     ax.add_patch(Circle((0, 0), EARTH_RADIUS_KM * 1e3,
140                        color='steelblue', zorder=5))
141
142     # Trayectorias
143     for (etq, df), color in zip(fuentes, PALETA):
144         ax.plot(df['x'].values, df['y'].values,
145               lw=1.2, color=color, label=etq)
146
147     # L mites cuadrados
148     all_x = np.concatenate([df['x'].values for _, df in fuentes])
149     all_y = np.concatenate([df['y'].values for _, df in fuentes])
150     cx = (all_x.min() + all_x.max()) / 2
151     cy = (all_y.min() + all_y.max()) / 2
152     hs = max(all_x.max() - all_x.min(), all_y.max() - all_y.min()) / 2 * 1.08
153     ax.set_xlim(cx - hs, cx + hs)
154     ax.set_ylim(cy - hs, cy + hs)
155     ax.set_aspect('equal', 'box')
156
157     ax.set_xlabel('x (m)')
158     ax.set_ylabel('y (m)')
159     ax.set_title('Comparaci n de trayectorias orbitales')
160     ax.tick_params(axis='x', rotation=45)
161     ax.legend()
162     fig.tight_layout()
```

```

163     fig.savefig(salida, dpi=dpi, bbox_inches='tight')
164     print(f"\n[comparar_orbitas] Imagen guardada: {salida}")
165     plt.close(fig)
166
167 def parse_args():
168     p = argparse.ArgumentParser(
169         description='Compara trayectorias orbitales lunares.',
170         formatter_class=argparse.RawDescriptionHelpFormatter,
171         epilog=__doc__
172     )
173     p.add_argument(
174         '--metodos', nargs='+', required=True,
175         metavar='M TODO',
176         help='Uno o m s m todos: rk4, euler, csv:<ruta>. Ej: --metodos rk4 euler csv:luna.csv'
177     )
178     p.add_argument('--dt', type=float, default=60.0,
179                    help='Paso de tiempo en segundos (default: 60)')
180     p.add_argument('--duracion', type=float, default=27.3,
181                    help='Duraci n en d as (default: 27.3)')
182     p.add_argument('--salida', type=str, default='resultados/
183 comparacion_orbital.png',
184                    help='Ruta de la imagen de salida (default: resultados/
185 comparacion_orbital.png)')
186     p.add_argument('--dpi', type=int, default=150,
187                    help='Resoluci n de la imagen (default: 150)')
188     p.add_argument('--no-exportar-csv', dest='exportar_csv', action='store_false',
189                    help='Desactiva la exportaci n de CSV por m todo (activada por
190 defecto)')
191     return p.parse_args()
192
193 def main():
194     args = parse_args()
195     print(f"[comparar_orbitas] Fuentes: {args.metodos}")
196     fuentes = resolver_fuentes(args.metodos, args.dt, args.duracion,
197                               exportar_csv=args.exportar_csv)
198     graficar(fuentes, salida=args.salida, dpi=args.dpi)
199
200 if __name__ == '__main__':
201     main()

```

7.3. Código trayectoria Orion

```

1 import argparse
2 import os
3 import sys
4
5 import numpy as np
6 import pandas as pd
7
8 import matplotlib
9 matplotlib.use('Qt5Agg')
10 import matplotlib.pyplot as plt
11 from matplotlib.patches import Circle
12 from matplotlib.backends.backend_qt5agg import FigureCanvasQTAff as FigureCanvas
13 from matplotlib.backends.backend_qt5agg import NavigationToolbar2QT as
14     NavigationToolbar
15
16 from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
17                               QHBoxLayout, QLabel, QLineEdit, QPushButton,
18                               QSlider, QMessageBox)
19
20 from PyQt5.QtCore import Qt
21
22 from discretizaciones import INTEGRADORES
23
24 CONST_G = 6.674e-11

```

```
23 M_T      = 5.972e24
24 M_L      = 7.342e22
25 EARTH_RADIUS_KM = 6371
26 MOON_RADIUS_KM = 1737
27
28 PALETA = ['#1f77b4', '#d62728', '#2ca02c', '#9467bd', '#8c564b', '#e377c2']
29
30
31 def gravitational_acceleration(x, y, pos_luna):
32     r2_t = x**2 + y**2
33     r_t = r2_t**0.5
34     if r_t == 0: ax_t = ay_t = 0.0
35     else:
36         a_t = CONST_G * M_T / r2_t
37         ax_t = -a_t * (x / r_t)
38         ay_t = -a_t * (y / r_t)
39
40     dx = x - pos_luna[0]
41     dy = y - pos_luna[1]
42     r2_l = dx**2 + dy**2
43     r_l = r2_l**0.5
44     if r_l == 0: ax_l = ay_l = 0.0
45     else:
46         a_l = CONST_G * M_L / r2_l
47         ax_l = -a_l * (dx / r_l)
48         ay_l = -a_l * (dy / r_l)
49     return (ax_t + ax_l, ay_t + ay_l)
50
51 def derivadas(estado, pos_luna):
52     x, y, vx, vy = estado
53     ax, ay = gravitational_acceleration(x, y, pos_luna)
54     return np.array([vx, vy, ax, ay])
55
56 def cargar_datos_luna(ruta: str) -> np.ndarray:
57     df = pd.read_csv(ruta, low_memory=False)
58     col_map = {}
59     for col in df.columns:
60         cu = col.strip().upper()
61         if cu in ('X_LUNA', 'X'): col_map[col] = 'x'
62         elif cu in ('Y_LUNA', 'Y'): col_map[col] = 'y'
63     df = df.rename(columns=col_map)
64     for c in ('x', 'y'): df[c] = pd.to_numeric(df[c], errors='coerce')
65     df = df.dropna(subset=['x', 'y'])
66     if df['x'].abs().max() < 1e7:
67         df['x'] *= 1e3
68         df['y'] *= 1e3
69     return df[['x', 'y']].to_numpy(), df[['x', 'y']].reset_index(drop=True)
70
71 def simular(nombre: str, dt: float, duracion_dias: float,
72             estado_inicial: np.ndarray, datos_luna: np.ndarray) -> pd.DataFrame:
73     integrador = INTEGRADORES[nombre]
74     pasos = int(duracion_dias * 86400.0 / dt)
75     n_luna = len(datos_luna)
76     estado = estado_inicial.copy()
77     t = 0.0
78     registros = np.empty((pasos + 1, 5))
79     registros[0] = [t, *estado]
80
81     for i in range(pasos):
82         pos_luna = datos_luna[-1] if i >= n_luna else datos_luna[i]
83         f = lambda t, s: derivadas(s, pos_luna)
84         estado = integrador(f, t, estado, dt)
85         t += dt
86         registros[i + 1] = [t, *estado]
87     return pd.DataFrame(registros, columns=['time_s', 'x', 'y', 'vx', 'vy'])
88
89 def cargar_csv_trayectoria(ruta: str) -> pd.DataFrame:
90     with open(ruta) as f: lineas = f.readlines()
91     sep = ';' if any('; ' in l for l in lineas[:5]) else ','
92     if sep == ';':
```



```
93     df = pd.read_csv(ruta, sep=';', decimal=',', header=None,
94                      names=['time_s', 'x', 'y', 'z', 'vx', 'vy', 'vz'])
95     else:
96         header_idx = None
97         for i, linea in enumerate(lineas):
98             partes = [p.strip().upper() for p in linea.split(',')]
99             if 'X' in partes and 'Y' in partes:
100                 header_idx = i
101                 break
102         if header_idx is None: sys.exit(1)
103         df = pd.read_csv(ruta, sep=',', header=header_idx, low_memory=False)
104         col_map = {}
105         for col in df.columns:
106             cu = col.strip().upper()
107             if cu in ('JDTDB', 'TIME_S', 'TIME'): col_map[col] = 'time_s'
108             elif cu == 'X': col_map[col] = 'x'
109             elif cu == 'Y': col_map[col] = 'y'
110         df = df.rename(columns=col_map)
111         for c in ('x', 'y'): df[c] = pd.to_numeric(df[c], errors='coerce')
112         df = df.dropna(subset=['x', 'y'])
113         if df['x'].abs().max() < 1e7:
114             df['x'] *= 1e3
115             df['y'] *= 1e3
116         return df
117
118 def resolver_fuentes(metodo, dt, duracion, estado_inicial):
119     resultados = []
120     if metodo.startswith('csv:'):
121         ruta = metodo[4:]
122         if os.path.exists(ruta):
123             df = cargar_csv_trayectoria(ruta)
124             resultados.append((os.path.splitext(os.path.basename(ruta))[0], df,
125                               None))
126     elif metodo in INTEGRADORES:
127         ruta_luna = f"resultados/resultado_{metodo.lower()}.csv"
128         if os.path.exists(ruta_luna):
129             datos_luna_arr, df_luna = cargar_datos_luna(ruta_luna)
130             df = simular(metodo, dt, duracion, estado_inicial, datos_luna_arr)
131             resultados.append((metodo.upper(), df, df_luna))
132     return resultados
133
134 class VentanaOrbital(QMainWindow):
135     def __init__(self, args):
136         super().__init__()
137         self.args = args
138         self.fuentes = []
139
140         self.setWindowTitle("Simulador Orbital Interactiva (Zoom Habilitado)")
141         self.resize(950, 850)
142
143         widget_central = QWidget()
144         self.setCentralWidget(widget_central)
145         layout_principal = QVBoxLayout(widget_central)
146
147         layout_controles = QHBoxLayout()
148         layout_controles.addWidget(QLabel("extra_v[0] (vx, m/s):"))
149         self.input_v0 = QLineEdit("-633")
150         self.input_v0.setFixedWidth(80)
151         layout_controles.addWidget(self.input_v0)
152
153         layout_controles.addWidget(QLabel("extra_v[1] (vy, m/s):"))
154         self.input_v1 = QLineEdit("-36")
155         self.input_v1.setFixedWidth(80)
156         layout_controles.addWidget(self.input_v1)
157
158         self.btn_recalcular = QPushButton("Recalcular")
159         self.btn_recalcular.setStyleSheet(
160             "background-color: #007ACC; color: white; font-weight: bold; padding: 5
161             px;")
162         self.btn_recalcular.clicked.connect(self.ejecutar_recalculo)
```

```
161 layout_controles.addWidget(self.btn_recalcular)
162 layout_controles.addStretch()
163 layout_principal.addLayout(layout_controles)
164
165 self.fig, self.ax = plt.subplots(figsize=(7, 7))
166 self.canvas = FigureCanvas(self.fig)
167
168 self.toolbar = NavigationToolbar(self.canvas, self)
169 layout_principal.addWidget(self.toolbar)
170 layout_principal.addWidget(self.canvas)
171
172 layout_slider = QHBoxLayout()
173 self.label_tiempo = QLabel("Paso: 0 / 0")
174 self.label_tiempo.setFixedWidth(150)
175 layout_slider.addWidget(self.label_tiempo)
176
177 self.slider_tiempo = QSlider(Qt.Horizontal)
178 self.slider_tiempo.setMinimum(0)
179 self.slider_tiempo.setMaximum(100)
180 self.slider_tiempo.setEnabled(False)
181 self.slider_tiempo.valueChanged.connect(self.actualizar_marcador_tiempo)
182 layout_slider.addWidget(self.slider_tiempo)
183 layout_principal.addLayout(layout_slider)
184
185 self.marcadores_moviles = []
186 self.ejecutar_recalculo()
187
188 def ejecutar_recalculo(self):
189     try:
190         ev0 = float(self.input_v0.text())
191         ev1 = float(self.input_v1.text())
192     except ValueError:
193         QMessageBox.critical(self, "Error de formato",
194                               "Introduce numeros decimales validos.")
195         return
196
197     x0 = -52409.924647197156 * 1000
198     y0 = -48671.635720228245 * 1000
199     vx0 = -1.22735334971968 * 1000 + ev0
200     vy0 = -2.34643943970753 * 1000 + ev1
201     estado_inicial = np.array([x0, y0, vx0, vy0])
202
203     self.fuentes = resolver_fuentes(
204         self.args.metodo, self.args.dt, self.args.duracion, estado_inicial
205     )
206
207     if not self.fuentes:
208         self.ax.clear()
209         self.ax.text(0.5, 0.5, "Sin datos validos.", ha='center', va='center')
210         self.canvas.draw()
211         self.slider_tiempo.setEnabled(False)
212         return
213
214     max_pasos = max(len(df_orion) for (_, df_orion, _) in self.fuentes) - 1
215     self.slider_tiempo.setMaximum(max_pasos)
216     self.slider_tiempo.setValue(0)
217     self.slider_tiempo.setEnabled(True)
218     self.dibujar_grafico_base()
219
220 def dibujar_grafico_base(self):
221     self.ax.clear()
222     self.marcadores_moviles.clear()
223
224     self.ax.add_patch(Circle((0, 0), EARTH_RADIUS_KM * 1e3,
225                               color='steelblue', zorder=5))
226     self.ax.plot([], [], 's', color='steelblue', label='Tierra')
227
228     for (etq, df_orion, df_luna), color in zip(self.fuentes, PALETA):
229         self.ax.plot(df_orion['x'].values, df_orion['y'].values,
230                     lw=1.2, color=color, label=f'Orion ({etq})', zorder=4)
```

```

231
232         if df_luna is not None:
233             self.ax.plot(df_luna['x'].values, df_luna['y'].values,
234                          lw=1.0, color=color, ls='--', alpha=0.4,
235                          label=f'Luna ({etq})', zorder=3)
236
237         marcador_p, = self.ax.plot([], [], 'o', color=color, ms=9, zorder=8)
238         marcador_l = None
239         if df_luna is not None:
240             marcador_l, = self.ax.plot([], [], 'X', color=color, ms=8, zorder
241 =8)
242
243         self.marcadores_moviles.append((df_orion, df_luna, marcador_p,
244 marcador_l))
245
246         all_x = np.concatenate([df['x'].values for _, df, _ in self.fuentes])
247         all_y = np.concatenate([df['y'].values for _, df, _ in self.fuentes])
248         for _, _, df_luna in self.fuentes:
249             if df_luna is not None:
250                 all_x = np.concatenate([all_x, df_luna['x'].values])
251                 all_y = np.concatenate([all_y, df_luna['y'].values])
252
253         cx = (all_x.min() + all_x.max()) / 2
254         cy = (all_y.min() + all_y.max()) / 2
255         hs = max(all_x.max() - all_x.min(), all_y.max() - all_y.min()) / 2 * 1.08
256         self.ax.set_xlim(cx - hs, cx + hs)
257         self.ax.set_ylim(cy - hs, cy + hs)
258         self.ax.set_aspect('equal', 'box')
259         self.ax.set_xlabel('x (m)')
260         self.ax.set_ylabel('y (m)')
261         self.ax.legend(fontsize=8, loc='upper right')
262         self.toolbar.update()
263
264         self.texto_info = self.ax.text(
265             0.02, 0.95, "", transform=self.ax.transAxes,
266             fontsize=9, verticalalignment='top',
267             bbox=dict(boxstyle='round', facecolor='white', alpha=0.85))
268
269         self.actualizar_marcador_tiempo(0)
270
271     def actualizar_marcador_tiempo(self, paso):
272         import datetime
273         self.label_tiempo.setText(f"Paso: {paso} / {self.slider_tiempo.maximum()}")
274         fecha_inicio = datetime.datetime(2026, 1, 1, 0, 0, 0)
275         texto_lineas = []
276
277         for df_orion, df_luna, marcador_p, marcador_l in self.marcadores_moviles:
278             idx_p = min(paso, len(df_orion) - 1)
279             x_act = df_orion['x'].iloc[idx_p]
280             y_act = df_orion['y'].iloc[idx_p]
281             marcador_p.set_data([x_act], [y_act])
282
283             if df_luna is not None and marcador_l is not None:
284                 idx_l = min(paso, len(df_luna) - 1)
285                 marcador_l.set_data([df_luna['x'].iloc[idx_l]],
286                                     [df_luna['y'].iloc[idx_l]])
287
288             vx = df_orion['vx'].iloc[idx_p]
289             vy = df_orion['vy'].iloc[idx_p]
290             t_segundos = df_orion['time_s'].iloc[idx_p]
291             magnitud_v = np.hypot(vx, vy)
292             angulo_deg = np.degrees(np.arctan2(vy, vx))
293             fecha_actual = fecha_inicio + datetime.timedelta(seconds=float(
294 t_segundos))
295             fecha_str = fecha_actual.strftime("%Y-%m-%d %H:%M:%S")
296
297             texto_lineas.append(
298                 f"--- Orion ---\n"
299                 f"Fecha/Hora: {fecha_str}\n"
300                 f"Mag. Vel: {magnitud_v:.2f} m/s\n"
301                 f"Angulo Vel: {angulo_deg:.2f} grados"

```

```
298         )
299
300         if self.texto_info:
301             self.texto_info.set_text("\n".join(texto_lineas))
302         self.canvas.draw_idle()
303
304
305 def parse_args():
306     p = argparse.ArgumentParser(description='Simulador interactivo con Zoom')
307     p.add_argument('--metodo', required=True, help='Ej: rk4')
308     p.add_argument('--dt', type=float, default=60.0)
309     p.add_argument('--duracion', type=float, default=10)
310     return p.parse_args()
311
312 def main():
313     args = parse_args()
314     app = QApplication(sys.argv)
315     ventana = VentanaOrbital(args)
316     ventana.show()
317     sys.exit(app.exec_())
318
319 if __name__ == '__main__':
320     main()
```